

Universidad de San Carlos de Guatemala
Facultad de Ingeniería
Escuela de Ciencias y Sistemas
Área de desarrollo de software
Software Avanzado

Lic. Marco Tulio Aldana Prillwitz
Xhunik Nikol Miguel Mutzutz



Proyecto Fase 1

Índice

1. Resumen	3
2. Objetivos	3
2.1 Objetivo General	3
2.2 Objetivos Específicos	3
3. Enunciado	4
3.1 Definiciones	4
3.2 Alcance funcional obligatorio (Fase 1)	4
3.2.1 Reglas de negocio mínimas (obligatorias)	4
3.3 Arquitectura 100% microservicios	5
3.3.1 Reglas estructurales	5
3.3.2 Lenguajes y frameworks permitidos	5
3.3.3 Comunicación	5
3.4 Servicios mínimos requeridos (Fase 1)	6
3.5 Persistencia y datos (obligatorio)	6
3.6 Redis (uso restringido: caché + fallback)	6
3.7 Integración entre equipos: Proveedor Externo (homologado y obligatorio)	7
3.7.1 Endpoint de publicación obligatorio (por cada grupo)	7
3.7.2 Consumo obligatorio (por cada grupo)	7
3.8 FX: conversión de moneda (API alternativa + caché/fallback)	7
3.9 Frontend (obligatorio)	8
3.10 Contratos (Swagger/OpenAPI)	8
3.11 CI (obligatorio)	8
3.11.1 Stage Build	8
3.11.2 Stage Test	9
3.11.3 Regla de versionado/tag obligatorio para ramas release	9
3.11.4 Delivery (recomendado en esta fase; obligatorio si el equipo usa registry)	9
3.12 Ejecución y despliegue (local y Google Cloud)	9
3.12.1 Ejecución local (obligatoria)	9
3.12.2 Ejecución en Google Cloud (obligatoria)	9
4. Entregables	10
4.1 Repositorio	10
4.2 Entregables mínimos (Fase 1)	10
5. Restricciones técnicas	11
6. Documentación	11
7. Consideraciones adicionales	11
7.1 Historial de desarrollo	11
7.2 Cálculo de nota individual	12
7.3 Integridad académica	12
8. Fecha y Modo de entrega	12

1. Resumen

En esta fase se desarrolla la **primera entrega del proyecto EconoMarket**, una plataforma **e-commerce modular** construida como **arquitectura 100% microservicios**, donde cada servicio tiene responsabilidad clara, contrato Swagger/OpenAPI, contenedor propio e imagen propia. El enfoque del proyecto contempla **≥6 microservicios** (catálogo, usuarios/auth, carrito/checkout, pagos, órdenes, notificaciones; u otro reparto equivalente), uso de **JWT**, consumo de **APIs externas entre equipos con resiliencia**, y conversión de monedas con **API pública alternativa con caché y fallback**.

La entrega debe poder ejecutarse en:

- **Local** usando **Docker Compose** (entorno reproducible), incluyendo health checks.
- **Google Cloud** (entorno reproducible documentado en README), preferiblemente mediante una **VM en Google Cloud** donde se instale Docker/Compose y se levante el stack completo con Compose; opcionalmente, el equipo puede desplegar algunos o todos los servicios con **Cloud Run**, si así lo decide y lo documenta.

2. Objetivos

2.1 Objetivo General

Diseñar y construir la base del proyecto EconoMarket como un conjunto de microservicios interoperable y desplegable, con autenticación JWT, persistencia en MS SQL Server, consumo resiliente de proveedores externos (entre grupos y FX), frontend con Vite y pipeline CI que ejecute pruebas y construya imágenes por microservicio..

2.2 Objetivos Específicos

1. Definir y aplicar reglas de negocio del dominio e-commerce (catálogo, carrito, checkout, notificaciones, pricing).
2. Implementar **≥6 microservicios** reales, cada uno en su contenedor e imagen.
3. Implementar un microservicio **Users/Auth** para emisión y validación de **JWT**.
4. Homologar integración entre equipos con el contrato de **Proveedor Externo**, consumible por todos.
5. Implementar conversión de moneda con **API pública alternativa** (FX), con caché TTL y fallback.
6. Persistir datos en **MS SQL Server** desde esta fase.
7. Incluir un **frontend** (Vite) que consuma REST.
8. Configurar **CI** para ejecutar pruebas y construir imágenes por microservicio, aplicando versionado/etiquetado en ramas **release/<vX>**.

3. Enunciado

3.1 Definiciones

Término	Definición
Cliente	Usuario final que navega productos, arma carrito y realiza compras.
Proveedor	Usuario que publica productos y recibe alertas de stock bajo.
Proveedor Externo	Catálogo publicado por otros grupos del curso que debe poder consumirse de forma interoperable.
Carrito	Listado persistente de productos + cantidades asociado a un cliente.
Checkout	Confirmación de compra y selección de forma(s) de pago.
Cartera (wallet)	Saldo interno del cliente utilizable como método de pago.
FX	Servicio de tipo de cambio para mostrar precios en diferentes monedas.

3.2 Alcance funcional obligatorio (Fase 1)

3.2.1 Reglas de negocio mínimas (obligatorias)

A) Precio visible al cliente (markup invisible)

El precio que ve el cliente se calcula como:

precio visible = precio base proveedor + 10%

Esta sumatoria se realiza **sin que el proveedor o el cliente lo observen como un concepto separado.**

B) Carrito persistente

- Si el cliente no ingresa por más de 1 día teniendo productos en el carrito, el sistema debe enviar un correo recordatorio.

- En esta fase, este comportamiento debe implementarse **sin jobs programados**: debe evaluarse en un evento de usuario (por ejemplo, login, apertura del carrito o navegación principal) comparando fechas y disparando el email si corresponde.

C) Checkout y pagos

- Se debe poder confirmar la compra desde el carrito.
- Antes de finalizar la compra, el cliente debe poder seleccionar pago con:
 - tarjeta débito/crédito, o
 - saldo en cartera, o
 - combinación de ambos.
- Se exige consistencia de reglas, validaciones y persistencia de la transacción (orden/pago) en MS SQL Server.

D) Notificaciones por correo (sin scheduler)

- Proveedor: notificar por correo cuando un producto tenga stock ≤ 10 .
- Cliente: al finalizar compra, enviar correo con productos comprados, cantidades y total.
- Debe quedar definido (y reflejado en el diseño) qué eventos disparan cada correo y cómo se evita "spam" (por ejemplo, registrar fecha/estado de notificación por evento).

3.3 Arquitectura 100% microservicios

3.3.1 Reglas estructurales

- El sistema debe tener **≥ 6 microservicios** reales.
- Todo vive en el repositorio: **<NUM_GRUPO>_SoftwareAvanzado_ProyectoFases**, pero:
 - **cada microservicio** debe tener su **Dockerfile**,
 - **cada microservicio** debe producir su **imagen**,
 - el CI debe construir una imagen **por microservicio**.

3.3.2 Lenguajes y frameworks permitidos

- Backend: **TypeScript, Golang, Python**.
- Regla obligatoria: deben usarse **al menos 3 lenguajes distintos** distribuidos entre microservicios.

3.3.3 Comunicación

- Interna entre microservicios: **GRPC**.
- Hacia frontend: **REST** (expuesto por gateway/BFF o por servicios públicos).

3.4 Servicios mínimos requeridos (Fase 1)

Para evitar soluciones “monolito disfrazado” y para cubrir el dominio, esta fase exige que existan, como mínimo, los siguientes servicios (los nombres son sugeridos; pueden variar si se documenta claramente):

1. **Users-auth-service (obligatorio)**: login/registro, emisión JWT, roles.
2. **catálogo-service**: productos, categorías, stock, proveedor y dueño.
3. **cart-checkout-service**: carrito persistente, checkout, cálculo de total/markup, creación de orden.
4. **orders-service o equivalente**: registro de órdenes/estado mínimo en MSSQL.
5. **payments-wallet-service o equivalente**: lógica de pago (tarjeta/cartera/mixto) y saldo de cartera.
6. **Notificaciones-service**: envío de correos (stock bajo, post-compra, recordatorio de carrito).
7. **integrations-service (obligatorio)**: consumo de catálogos de otros grupos + caché/fallback.
8. **fx-service (obligatorio)**: consumo de API pública alternativa de tipo de cambio + caché/fallback.

Nota: se permite incluir un **API Gateway/BFF** como servicio adicional para centralizar REST hacia frontend.

3.5 Persistencia y datos (obligatorio)

- DB obligatoria: **MS SQL Server**.
- Debe existir persistencia real (no memoria) al menos para:
 - usuarios/roles,
 - catálogo (productos/categorías/stock),
 - carrito y órdenes/pagos (o equivalente).

Ejecución:

- En el local debe existir un servicio MSSQL dentro del Docker Compose. En Google Cloud, el equipo puede:
 - ejecutar MSSQL como contenedor en la VM con Compose, o
 - usar un servicio administrado (si lo tienen disponible), documentando credenciales y red.

3.6 Redis (uso restringido: caché + fallback)

Redis se utiliza **únicamente** como caché/fallback para:

1. **Proveedor Externo** (catálogos de otros grupos)
2. **FX** (tasas de cambio)

No se permite usar Redis como base de datos principal ni como “source of truth” del dominio.

Ejecución:

- En local: Redis como servicio dentro del Compose.
- En Google Cloud: Redis puede correr como contenedor en la VM con Compose o usar un Redis administrado (documentado).

3.7 Integración entre equipos: Proveedor Externo (homologado y obligatorio)

3.7.1 Endpoint de publicación obligatorio (por cada grupo)

Cada grupo debe exponer públicamente este endpoint REST, sin JWT (solo lectura):

GET /external-catalog/v1/products

Debe incluir, como mínimo, los datos exigidos para cada producto: SKU, imagen, nombre, precio, stock, categoría, etiqueta “Proveedor Externo” y número de grupo.

Reglas mínimas de interoperabilidad:

- `externalLabel` debe ser exactamente “Proveedor Externo”.
- `groupNumber` debe ser el número del grupo dueño del catálogo.
El contrato OpenAPI del endpoint debe estar incluido en el repositorio.

3.7.2 Consumo obligatorio (por cada grupo)

Cada grupo debe consumir catálogos externos desde su **integrations-service**:

- Lista de proveedores externos por configuración (variables de entorno).
- Resiliencia: timeout + reintento simple + degradación elegante.
- Redis como fallback:
 - si el proveedor responde → actualizar caché y responder “LIVE”
 - si falla → responder desde caché (si existe) “CACHED FALLBACK”
si falla y no hay caché → “UNAVAILABLE”

3.8 FX: conversión de moneda (API alternativa + caché/fallback)

El sistema debe manejar precios internamente en quetzales y poder mostrar al cliente precios en monedas como USD/MXN/JPY u otras equivalentes según disponibilidad.

Proveedor externo homologado (API alternativa):

- **Endpoint Abierto:** <https://open.er-api.com/v6/latest/USD>

- Cambiar **USD** por otra base (ej. EUR):
<https://open.er-api.com/v6/latest/EUR>

Reglas mínimas del fx-service:

- Endpoint configurable por variables de entorno.
- Caché con TTL + manejo de rate-limits.
- Degradación elegante: si falla la API externa, usar última tasa cacheada o mostrar moneda base con aviso.

3.9 Frontend (obligatorio)

Frontend permitido: cualquiera soportado por **Vite** (Vue/React/...).

Debe incluir, como mínimo:

- navegación por categorías y listado de productos,
- carrito (agregar/quitar/actualizar),
- login/logout (JWT) y un flujo mínimo de checkout,
- Visualización de productos externos claramente etiquetados como “Proveedor Externo”.

Ejecución:

- En local: el frontend debe levantarse como parte del stack (servicio más en Compose o servirse desde un contenedor).
- En la nube: el frontend debe ser accesible desde fuera. El equipo puede:
 - levantarlo en la VM como contenedor (recomendado por simplicidad),
 - o desplegarlo en Cloud Run / hosting estático, si lo documenta.

3.10 Contratos (Swagger/OpenAPI)

- Contratos de servicio: **Swagger/OpenAPI**.
- Debe existir al menos un OpenAPI por cada servicio que exponga REST (incluyendo el endpoint de Proveedor Externo y los endpoints consumidos por frontend).
- El versionado de endpoints debe ser explícito (por ejemplo `/v1/...`) para evitar rupturas.

3.11 CI (obligatorio)

El repositorio debe incluir un pipeline de CI que, como mínimo:

3.11.1 Stage Build

- Construya las imágenes Docker de **cada microservicio** usando sus Dockerfiles (multi-stage recomendado).
- Construya el contenedor del frontend si se entrega como imagen.

3.11.2 Stage Test

- Ejecute pruebas unitarias (mínimas) de forma automática y falle el pipeline si alguna prueba falla.
- Las pruebas deben aplicar a servicios backend y al frontend cuando corresponda.

3.11.3 Regla de versionado/tag obligatorio para ramas release

Cuando la rama de destino sea `release/<vX>`:

- el pipeline debe construir la imagen de **cada microservicio** con el tag:

`[nombre-servicio]:<vX>`

3.11.4 Delivery (recomendado en esta fase; obligatorio si el equipo usa registry)

Si el equipo decide utilizar un registry (recomendado por prácticas CI/CD), el pipeline debe:

- autenticarse usando secretos del CI (no credenciales en el repo),
- publicar imágenes para todos los servicios (backend y frontend si aplica),
- Documentar en README cómo se ejecuta localmente y cómo se levanta en la VM en Google Cloud.

3.12 Ejecución y despliegue (local y Google Cloud)

3.12.1 Ejecución local (obligatoria)

Debe existir un `docker-compose.yml` (o `compose.yaml`) que levante:

- todos los microservicios,
- MS SQL Server,
- Redis,
- frontend,
- (opcional) gateway/BFF, incluyendo **health checks**.

El README debe incluir comandos exactos para:

- build y up
- variables de entorno necesarias (`.env` de ejemplo) cómo validar que el sistema está listo (health o endpoints)

3.12.2 Ejecución en Google Cloud (obligatoria)

Debe existir un entorno reproducible en Google Cloud, documentado en README.

Requisito mínimo recomendado:

- Crear una **VM** en Google Cloud,
- instalar Docker y Docker Compose,
- desplegar el proyecto usando el mismo Compose,
- exponer únicamente los puertos necesarios (ver siguiente punto).

Exposición de puertos (regla de seguridad mínima):

- Solo se permite exponer públicamente:
 - frontend (HTTP/HTTPS),
 - y el gateway/BFF o API pública necesaria para el frontend.
- Redis y MS SQL Server **no deben exponerse** públicamente (solo accesibles dentro de la red de contenedores).

Opción adicional: Cloud Run (opcional)

El equipo puede desplegar algunos o todos los servicios en Cloud Run, siempre que:

- mantenga los mismos contratos,
- documente claramente endpoints, configuración y red,
- y garantice que el sistema es verificable por el auxiliar.

4. Entregables

4.1 Repositorio

- Enlace del repositorio en UEDi.
- Estructura sugerida:
 - `services/<service-name>/...`
 - `frontend/...`
 - `docs/...`
 - `contratos/...`
 - `docker-compose.yml / compose.yml`
 - `README.md`
 - `INTEGRANTES.md`

4.2 Entregables mínimos (Fase 1)

- ≥6 microservicios ejecutables, cada uno con responsabilidad clara.
- Dockerfile por servicio (multi-stage recomendado).
- docker-compose de desarrollo con health checks.
- Contratos Swagger/OpenAPI por servicio (cuando aplique REST).
- Pipeline CI con pruebas unitarias y build de imágenes por microservicio.
- Evidencias en `/docs` (requisitos priorizados, trazabilidad, vistas, riesgos/mitigaciones, ADRs).

- Evidencia de ejecución en Google Cloud (VM con Compose) documentada en README.

Documento de requerimientos (obligatorio): Cada grupo debe completar y entregar un documento de requerimientos del proyecto, incluyendo **requerimientos estratégicos, funcionales, no funcionales, técnicos, riesgos y restricciones**. El formato se encuentra disponible en UEDi con el nombre de “**Documentación Requerimientos.docx**”; el equipo deberá basarse en esa plantilla, llenarla con la información del proyecto y agregar el archivo final al repositorio dentro de **/docs** en formato PDF.

5. Restricciones técnicas

- Nube a usar: **Google Cloud**.
- Backend: **TypeScript, Golang, Python**.
- Frontend: cualquiera soportado por **Vite**.
- API interna: **gRPC**, excepto frontend (REST).
- DB: **MS SQL Server**.
- Contratos: **Swagger/OpenAPI**.
- Seguridad: **JWT**.
- Redis: solo caché/fallback para Proveedor Externo y FX.
- Orquestación para esta fase: **Docker Compose** (local y en VM).

6. Documentación

En **/docs** deben incluirse evidencias y decisiones del diseño para Fase 1, incluyendo como mínimo:

- Planning
- requisitos funcionales y no funcionales priorizados y decisiones tomadas,
- casos de uso y trazabilidad a microservicios,
- vistas de arquitectura (lógica, proceso/interacción, despliegue alto nivel),
- riesgos y mitigaciones (especialmente integraciones externas, timeouts, idempotencia, consistencia eventual),
- ADRs relevantes,
- evidencias del pipeline CI (capturas/logs o descripción reproducible).

7. Consideraciones adicionales

7.1 Historial de desarrollo

Se revisará el historial de commits para verificar avance incremental (no realizado en un único commit).

7.2 Cálculo de nota individual

La nota individual se obtiene a partir de la nota grupal y un índice de aporte calculado con Git Fame. Para efectos de este curso, **los equipos son siempre de 5**, por lo que el “aporte esperado” por integrante es **20** en una escala de 0 a 100.

Valores (0 a 100) tomados de Git Fame

- **LOC**: aporte por líneas de código
- **COMMITTS**: aporte por commits
- **FILES**: aporte por archivos modificados

Cálculos

- $\text{ÍNDICE} = 0.6 * \text{LOC} + 0.2 * \text{COMMITTS} + 0.2 * \text{FILES}$
- $\text{FACTOR} = \text{MIN}(\text{ÍNDICE} / 20, 1)$
- $\text{NOTA INDIVIDUAL} = \text{NOTA GRUPAL} * (0.6 + 0.4 * \text{FACTOR})$

Campo	Fórmula
ÍNDICE	$0.6 * \text{LOC} + 0.2 * \text{COMMITTS} + 0.2 * \text{FILES}$
FACTOR	$\text{MIN}(\text{ÍNDICE} / 20, 1)$
NOTA INDIVIDUAL	$\text{NOTA GRUPAL} * (0.6 + 0.4 * \text{FACTOR})$

7.3 Integridad académica

La copia total o parcial implica calificación de 0 y será reportada a la escuela de ciencias y sistemas.

8. Fecha y Modo de entrega

- **Fecha límite:** viernes 12 de diciembre, 23:59.
- **Entrega:** enlace del repositorio GitHub en la plataforma UEDi. El repositorio debe ser accesible e incluir todos los entregables y tags requeridos.